FinSecure Pay: Design Document

Automating a Cloud-Native DevOps Pipeline | DevOps, Cloud and AI Capstone (individual)
GitHub: https://github.com/sakthivel0301/finsecure-pay | Docker Hub: https://hub.docker.com/r/sakthivel0301/finsecure-pay

1. Purpose and scope

FinSecure Pay is a small fintech that needs a fast, repeatable and reliable way to ship a transaction validation API. This document describes the solution that takes the application from source code to a production-style deployment on managed Kubernetes with minimal manual intervention. It covers the architecture, the main tool choices and trade-offs, security and recovery design, monitoring and logging, and one practical AI use case.

The application is a Spring Boot (Java 17, Maven) service with two endpoints: `GET /health` and `POST /validate`. Validation rejects negative or zero amounts, identical source and destination accounts, unsupported currencies and amounts above the configured maximum (100,000), and returns every error found. It is covered by 16 JUnit 5 tests (unit and MockMvc) with JaCoCo coverage, and exposes Actuator health probes and Prometheus metrics, including a custom counter of valid and invalid transactions.

2. End-to-end architecture

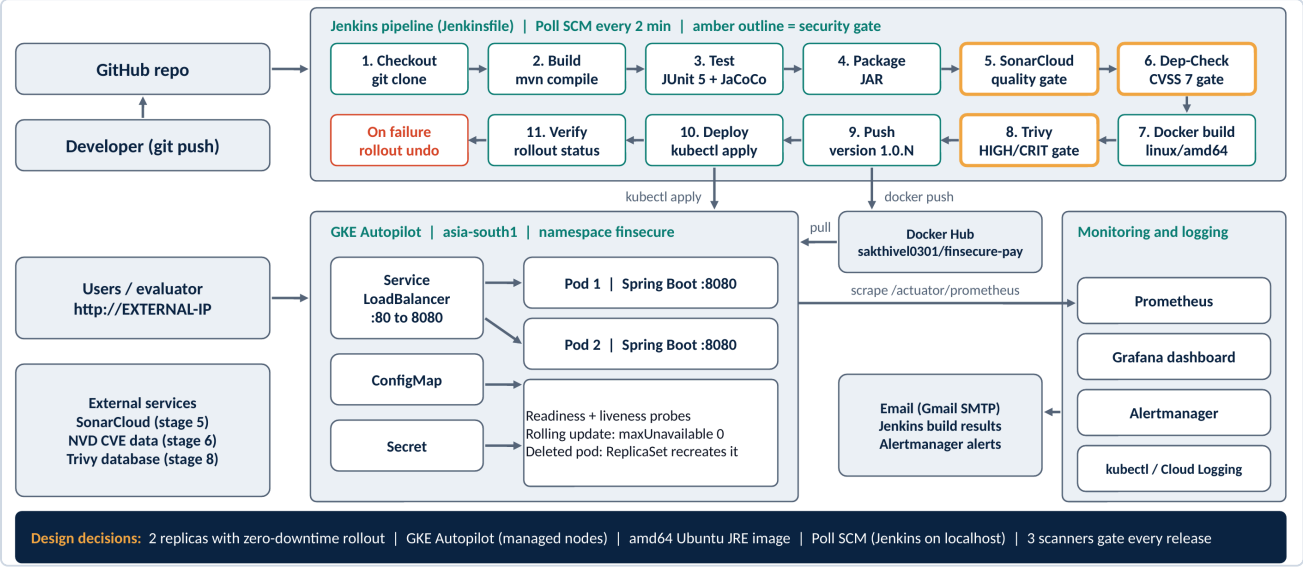


Figure 1. One commit from developer to production, with the three security gates, rollout verification and observability.

Flow in words

- A developer pushes to GitHub. Jenkins polls the repository every two minutes (Poll SCM) and runs the `Jenkinsfile`.
- Stages 1 to 4 check out, build, test and package. Stages 5, 6 and 8 are security gates: SonarCloud quality gate, OWASP Dependency-Check (fails at CVSS 7 or above) and Trivy (fails on fixable HIGH or CRITICAL findings).
- Stage 7 builds a `linux/amd64` image, stage 9 pushes it to Docker Hub tagged `1.0.<build number>`, stage 10 updates the Deployment image and applies the manifests to GKE, and stage 11 waits for the rollout. If the rollout fails, the pipeline runs `kubectl rollout undo` automatically.
- Users reach the application through a LoadBalancer Service that fronts two pods. Prometheus scrapes `/actuator/prometheus`, Grafana visualises it, Alertmanager and Jenkins send email through Gmail SMTP, and logs are read with `kubectl logs` or Cloud Logging.

3. Tool choices and justification

Area	Choice	Why
Source control	Git, GitHub	Recommended tool; commit history and a public link for the evaluator.
Application and tests	Spring Boot 4.1.2, Maven, JUnit 5, JaCoCo	Recommended stack; Actuator and Micrometer give health probes and metrics with little code. Boot was upgraded from 3.3.x to clear known CVEs flagged by the scanners.
Containers	Docker multi-stage build, Docker Hub	Small runtime image, non-root user, built-in HEALTHCHECK. Ubuntu-based `eclipse-temurin:17-jre` runtime, because no Alpine JRE exists for arm64 on the Apple Silicon build machine.
Orchestration	GKE Autopilot (asia-south1)	Approved managed Kubernetes. Autopilot removes node management, which suits a small team; the trade-off is less control over nodes and a billing account requirement.
CI/CD	Jenkins with a declarative Jenkinsfile	Recommended tool. Runs on the developer machine for the capstone. A webhook cannot reach `localhost`, so Poll SCM triggers builds; in production a webhook or GitOps would replace it.
Quality and security	SonarCloud, OWASP Dependency-Check, Trivy, gitleaks	Covers code, dependencies and image, so a release is blocked at the stage that found the problem. SonarCloud is the hosted variant of SonarQube and needs no server to run.
Secrets	Jenkins Credentials, Kubernetes Secrets	Nothing sensitive in Git. Jenkins masks bound values in logs. A cloud secret manager is a future step.
Monitoring	Prometheus, Grafana, Alertmanager	Recommended tools; run with Docker Compose on the same machine to limit cloud cost.
Logging	kubectl logs, Cloud Logging (cloud-native)	Approved cloud-native option; avoids the cost and operations of running an ELK stack for two pods.
Notifications	Jenkins Email Extension, Gmail SMTP	Recommended tool; the same SMTP account serves Alertmanager.

4. Key design decisions and trade-offs

- **Two replicas with a rolling update (maxUnavailable 0, maxSurge 1).** A new pod must become Ready before an old one is removed, so a release causes no downtime. Cost: briefly three pods run during a rollout.
- **Configuration outside the image.** A ConfigMap holds log level and version; the same image runs unchanged in every environment.
- **Probes on Actuator endpoints.** Readiness keeps traffic away from a pod that is starting; liveness restarts a stuck one.
- **Gates before side effects.** All three scanners run before the push and deploy stages, so a vulnerable build never reaches the registry or the cluster.
- **Unique, traceable image tags.** Each build is tagged with its build number and the deployment records a change-cause, so rollback targets a known-good revision rather than `latest`.
- **Reproducible, low-cost operation.** Monitoring stays outside the cluster and the cluster is deleted after evidence is captured; production would run Prometheus in-cluster or use a managed service.

5. Security considerations

- **Shift-left gates.** SonarCloud (`sonar.qualitygate.wait=true`) checks code quality; Dependency-Check fails the build at CVSS 7 or above using a cached NVD database and an API key; Trivy scans the built image for HIGH and CRITICAL findings, ignoring those without a fix.
- **Demonstrated block and remediation.** A known-vulnerable dependency made Dependency-Check fail the build; upgrading Spring Boot cleared the findings. A base-image finding made Trivy fail until the Dockerfile upgraded the affected package. A false positive was documented in a dated suppression file instead of disabling the gate.
- **Secrets.** Docker Hub, SonarCloud, NVD and SMTP credentials live in Jenkins Credentials and are bound only to the steps that need them; logs show `****`. `.gitignore` and gitleaks keep secrets out of the repository, and Kubernetes Secrets (never printed as YAML) carry runtime values.
- **Runtime hardening.** The container runs as a non-root user with a minimal JRE image; the Service exposes only the application port.

6. Recovery considerations

- **Pod failure.** The Deployment's ReplicaSet recreates a deleted or crashed pod automatically; the second replica keeps serving meanwhile. Demonstrated by deleting a pod and watching it return to Ready.
- **Bad release.** Stage 11 checks rollout status and runs `rollout undo` on failure. Rollout history with change-cause annotations allows a manual rollback to any earlier revision.
- **Failed tests or scans.** The pipeline stops before the push and deploy stages, so the running version is untouched.
- **Detection.** Alerts for instance down, HTTP 5xx errors and a high invalid-transaction rate notify the operator by email, and Jenkins emails the result of every build.
- **Known gaps.** Jenkins and the monitoring stack run on one machine, a single cluster in one region has no disaster-recovery site, and there is no autoscaling. These are listed under future improvements.

7. Monitoring, alerting and logging

Prometheus scrapes the application every few seconds. The Grafana dashboard "FinSecure Pay - Operations" shows request rate, latency, HTTP status codes, valid versus invalid transactions, instance up state and JVM memory. Three alert rules are defined in `alert-rules.yml` (instance down, 5xx rate, invalid-transaction rate); the invalid-transaction alert is triggered on purpose with a script that posts bad transactions, and Alertmanager emails the result. For troubleshooting, application logs are read with `docker logs` locally, `kubectl -n finsecure logs -l app=finsecure-pay` in the cluster, and in Cloud Logging, filtered by namespace and severity.

8. AI use case: LLM-assisted incident triage

Question	Answer
Problem and why AI	When an alert fires, the on-call engineer must gather logs, metrics and recent deployments, then decide what changed. This is slow and repetitive, and the evidence is mostly unstructured text, which is a good fit for a language model. A rule-based script cannot summarise free-form logs or relate them to a deployment.
Inputs	The Alertmanager payload (alert name, labels, time), the last N minutes of application logs, key Prometheus metrics around the alert, `kubectl rollout history` and the latest Jenkins build and commit message, plus a fixed runbook prompt.
Output	A short email summary: probable cause ranked with evidence, affected component, the suggested check or rollback command, and a confidence note.
Benefits	Faster triage and lower mean time to resolution, consistent first-response quality, less alert fatigue, and a natural-language record attached to each incident.

Risks and governance	The model can be wrong or invent causes, so it only suggests and a human decides; it never runs changes. Logs are masked for account numbers, tokens and emails before leaving the cluster, ideally using an approved or self-hosted model. Prompts and outputs are logged for audit, and answers must cite the log lines they rely on.
Integration	Alertmanager gets a webhook receiver pointing to a small triage service. The service collects context (Prometheus API, `kubectl`, Cloud Logging), masks it, calls the model and emails the summary through the existing SMTP path. The same service could later add anomaly detection on the metrics already collected, to alert before thresholds are crossed.

9. Lessons learned and future improvements

Lessons. Apple Silicon builds arm64 images while GKE needs amd64. Vulnerability data changes quickly, so every gate needs a documented remediation path and a way to record false positives. Cloud billing and cluster lifecycle must be planned, because a lapse took the cluster down and evidence has to be captured while it is up. Small configuration details (SMTP settings, Java versions, credential types) caused many failures.

Future improvements. GitHub webhooks and GitOps (Argo CD, Helm); ingress with TLS and autoscaling; a secret manager (Vault or Secret Manager); in-cluster monitoring and central logging; image signing and SBOMs; and the AI triage service described above.